

Quality Assurance Report

1. Executive Summary

The codebase for the ExpenseTrack application demonstrates a well-structured modular design and aligns generally with the Product Requirements Document (PRD) and architectural specifications. However, the presence of critical security and functional vulnerabilities prevents the system from being production-ready. Most notably, the backend API exposes user passwords in plaintext during login and uses insecure password hashing (SHA-256 instead of bcrypt), while also lacking proper authorization checks on sensitive operations. Additionally, the frontend stores JWT tokens in localStorage, increasing exposure to XSS attacks.

Due to these high-severity issues—particularly those involving data security and user authorization—the build FAILED the quality assurance evaluation. Immediate remediation is required before deployment to ensure data integrity, user privacy, and system reliability.

2. Quality Metrics

Metric	Count
Total Bugs Found	7
Critical Issues	2
Total Tests Written	5

3. Service Reports

Backend API - Code Quality Score: 5/10

Bugs Found

Severity	Description	Suggested Fix

```
-|-----|
| Critical | The login endpoint returns a plain text password in the response body.
              | Remove password from the response model and ensure no
sensitive data is ever returned in API responses. |
| Critical | Expense creation endpoint does not validate category against predefined
list (Food, Transport, etc.). | Add category validation using a predefined enum or
list before saving to database. |
| Major    | Update and delete expense endpoints do not verify that the requesting
user owns the expense. | Add ownership check by comparing
expense.user_id with current_user.id before allowing modification. |
| Major    | Password hashing uses SHA-256 instead of bcrypt as specified in PRD and
security requirements. | Replace SHA-256 with bcrypt or equivalent adaptive
hashing algorithm like Argon2. |
| Minor    | Report endpoints do not filter data by authenticated user, risking data
leakage. | Ensure all report queries are scoped to request.user.id.
|
```

Test Cases Generated

Test Name	Target File	Description
test_login_returns_no_password	app/api/auth.py	Verifies that login response does not include password
test_category_validation_on_expense_create	app/api/expenses.py	Ensures only valid categories can be used when creating expenses
test_user_can_only_delete_own_expenses	app/api/expenses.py	Prevents users from deleting expenses they don't own

Frontend UI - Code Quality Score: 7/10

Bugs Found

Severity	Description
Suggested Fix	

| Major | JWT tokens are stored in localStorage which is vulnerable to XSS attacks.
| Use httpOnly cookies for token storage or implement secure in-memory storage with proper cleanup. |
| Minor | Date field does not default to current date as specified in PRD acceptance criteria. | Set default value of date input to new Date().toISOString().split('T')[0] on component mount. |

Test Cases Generated

Test Name	Target File	Description
test_expense_form_defaults_today_date	src/pages/AddExpense.jsx	Verifies that the date field defaults to current date
test_category_dropdown_has_valid_options	src/pages/AddExpense.jsx	Ensures category field uses predefined valid options

4. Recommendations

To bring the application to a production-ready state, the following actions are strongly recommended:

- Immediately fix the password exposure in the login response.
- Implement proper bcrypt password hashing in place of SHA-256.
- Add user ownership checks on all expense creation, update, and delete operations.
- Validate expense category against the allowed list (e.g., Food, Transport, Utilities, etc.).
- Store JWT tokens securely using httpOnly cookies or secure in-memory storage.
- Ensure all report endpoints are scoped to the current authenticated user.
- Add comprehensive error handling across all API routes.
- Implement consistent input validation on both frontend and backend layers to prevent invalid data entry.

Addressing these issues will significantly improve the security posture, functional correctness, and overall reliability of the ExpenseTrack application.